ICG202 Introduction to Computer Graphics

Assessment 3: Shader and Environment Design

Giada Arosio (A00168823)

Introduction

The project starts from a pre-built Unity environment, see figure 1. The scene consists of a small island with several elements, including a crypt structure, a fountain, an interactive chest that opens automatically when the player approaches, and a first-person player controller.

Figure 1

Original scene provided

The goal was to reimagine the environment provided around a chosen concept and enhance this base landscape by building a custom shader, redesigning the atmosphere and scene elements, and adding a special effect.

Concept

The chosen concept is a dark, horror-themed setting: an isolated island in the middle of the ocean, populated by spiders, wrapped in a cold, dark night.

The environment is deliberately underlit, relying on weak moonlight, small warm lanterns, and a strange red glow visible across the island, suggesting something unnatural beneath the ground.

Also, a chest gives the player the ability to burn the spiders once opened, simply by approaching them. This power is expressed visually through the dissolve shader developed for the spiders, while the chest's opening itself is accompanied by a particle effect and a concentrated light, designed to communicate the moment the power is acquired.

The following sections describe how each of these elements was built: the environment and atmosphere, the dissolve shader, and the special effects used to produce the final visual result.

Environment

Several existing elements in the scene were adjusted to better fit the chosen concept and create a stronger atmosphere, see Figure 2.

Figure 2

Overview play-mode. Vegetation, crypt and skybox.

Vegetation

Tree size was increased and additional rocks, with properly resized or added colliders, were placed around the island to make the terrain feel denser and more overgrown.

Crypt Orientation

The crypt was rotated to create a more suggestive framing when viewed from the player's starting direction. This makes the structure a clear visual focal point as the player progresses through the environment from the very beginning.

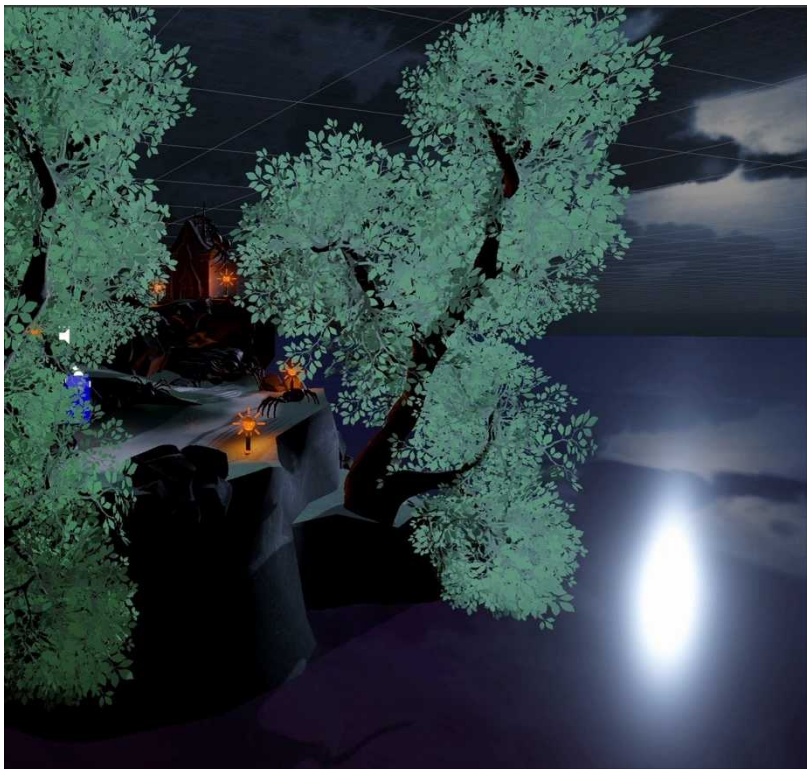
Skybox

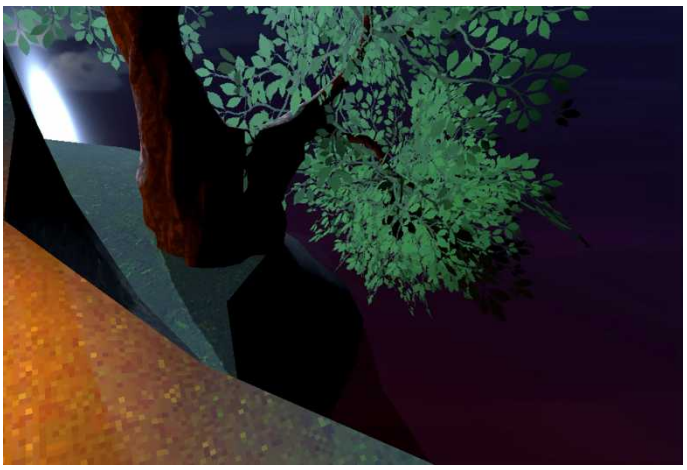
A night skybox, showing a moon hidden behind the clouds, was imported (Rpgwhitelock, 2024) and rotated so that the moon appears near the crypt and along the player's main path. This makes the moon feel like the main source of light, creating a suggestive atmosphere.

Water

Water was added in two areas of the environment. An ocean system with animated waves and reflections was added (CodeDog0, 2024) around the island to remove the empty background present in the original scene and make the environment feel more believable, see Figure 3.

Figure 3
Videos: animated ocean waves, reflections and red shades dettails.





From the same asset, water was also added to the fountain together with the included foam and movement effect, bringing life to an element that was previously static (Figure 4).

Figure 4
Video: fountain water movement, foam effect and red shades.

To maintain visual consistency with the horror theme, reddish water shades were incorporated into both the ocean and fountain water.

Lighting and atmosphere

Lighting was built around two contrasting sources. The Directional Light was set to recreate the cold tone of the moonlight, with its reflection visible on the ocean and shadows matching the direction of the moon (Figure 5).

Figure 5
Directional Light – Lighting and shadows.

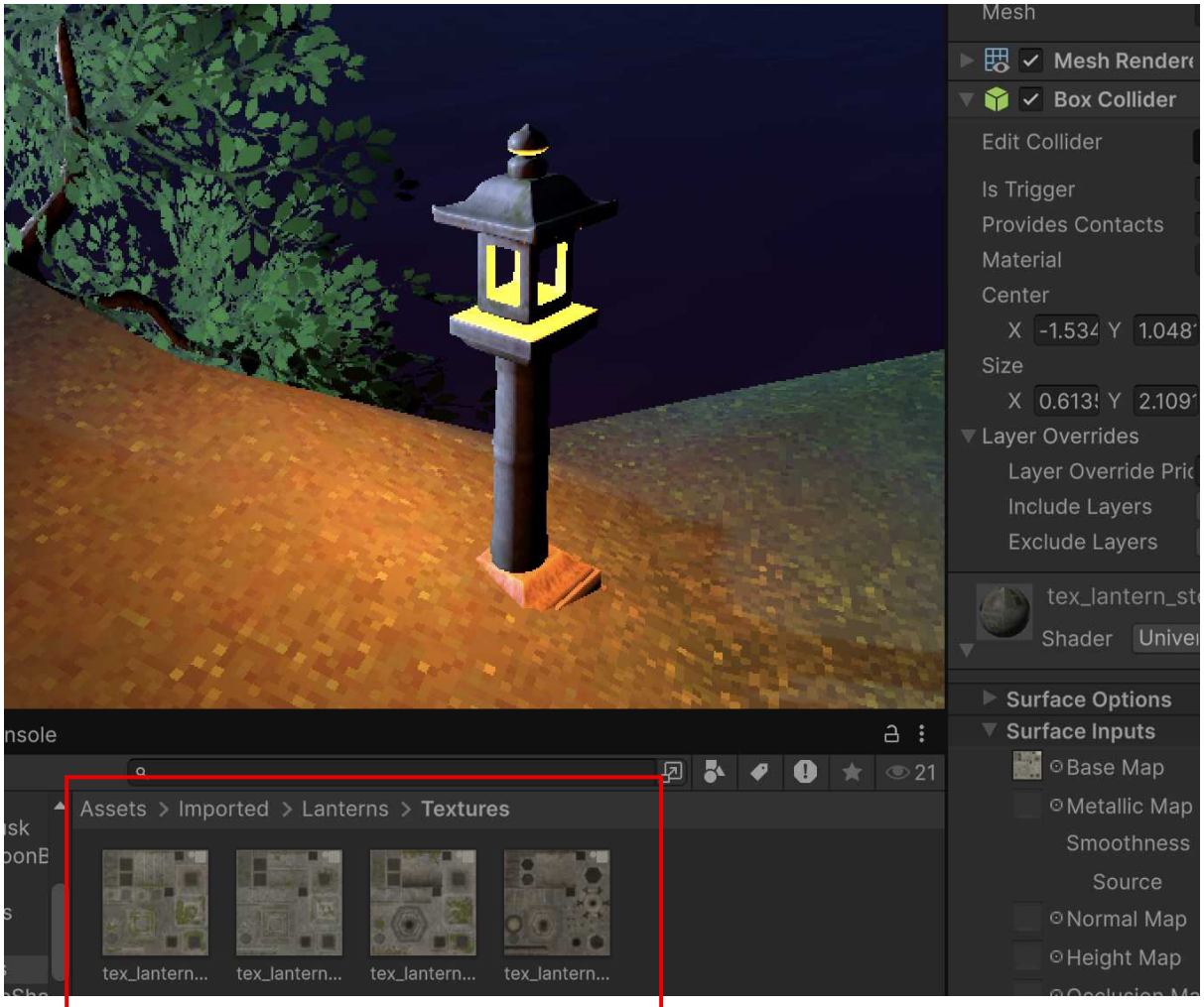
Against this cold light, a red-toned environment light was added, casting reflections across the scene. This light deliberately contrasts with the cold moonlight and creates the impression that an unnatural source of evil energy exists beneath the island (Figure 6). This effect is repeated in the reddish tones of the fountain and ocean water, as explained above, helping unify the visual style of the scene (Figure 3-4).

Figure 6
Environment Light – Example of before and after red tones.

Lanterns

Five lanterns (Tanuki Digital, 2019) were placed around the island, each with its own collider and a warm point light source, to highlight the player's path and the crypt. Also, four different texture variations were used to avoid repetition and make them look more realistic, see Figure 7.

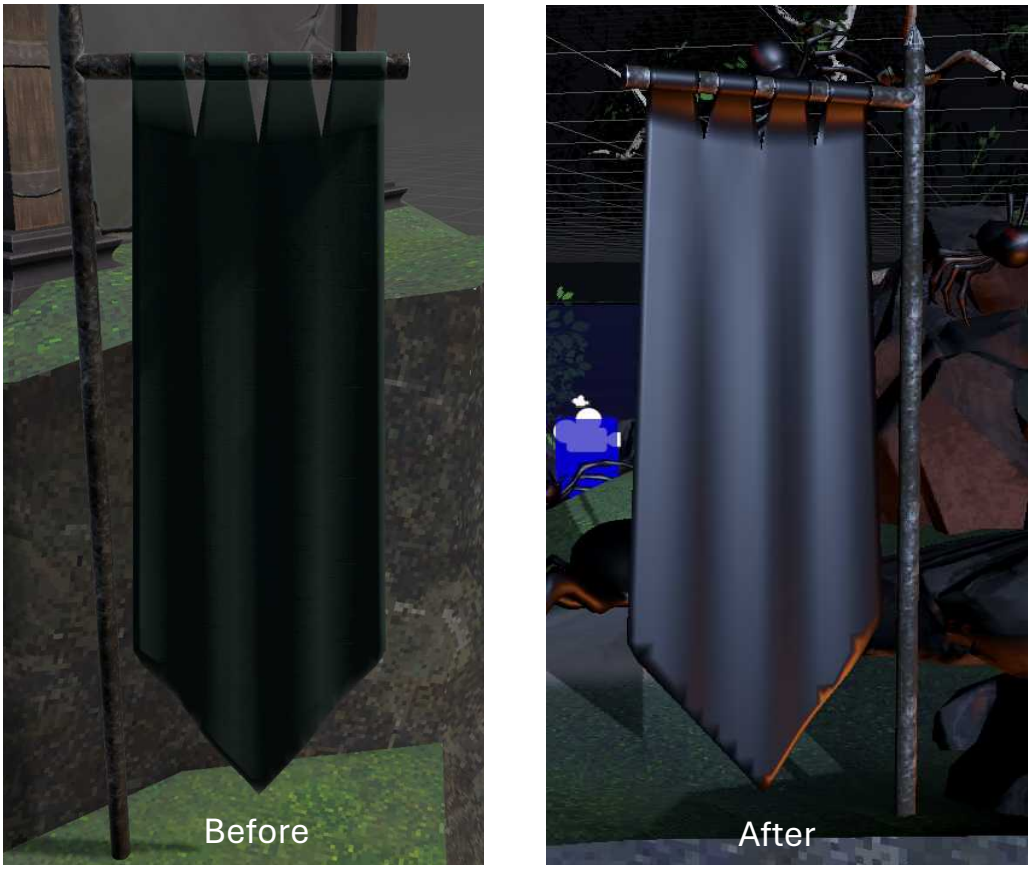
Figure 7
Lantern texture dettails.



Flag

The original flag material was replaced with the same texture used on the black widow spiders, see Figure 8. This was intended to suggest that the island belongs to the spiders and reinforce the overall theme of the environment.

Figure 8
Before and after - Flag texture dettails.



Camera and player starting position

The camera and player's starting position were moved so that the ocean, the fountain, and a spider resting on it are the first things visible when the game begins. Rather than pointing directly to a clear objective, this framing was chosen to place the

player in an open, slightly disorienting view, reinforcing the feeling of being alone and isolated on the island from the very beginning (Figure 9).

Figure 9
Before and after - Starting view.

Spiders

Multiple animated spiders were introduced into the environment as a threat. Each spider uses the black widow texture provided with the downloaded asset (Dante's Anvil, 2022) and a custom shader (see following section).

Moreover, to make the creatures feel more believable, three animation states were implemented:

- Idle: continuous looping animation (Figure 10)

  Figure 10
  Spider Idle animation.

- Scare: triggered when the player approaches without the power (Figure 11)

  Figure 11
  Spider Scare animation.

- Death: triggered when the spider is burned (Figure 12)

Figure 12
Spider Death and burn (dissolve shader effect) animation.

The spiders also contain two colliders: a physical collider that blocks player movement, and a trigger collider used to detect the player's approach and drive the burning interaction.

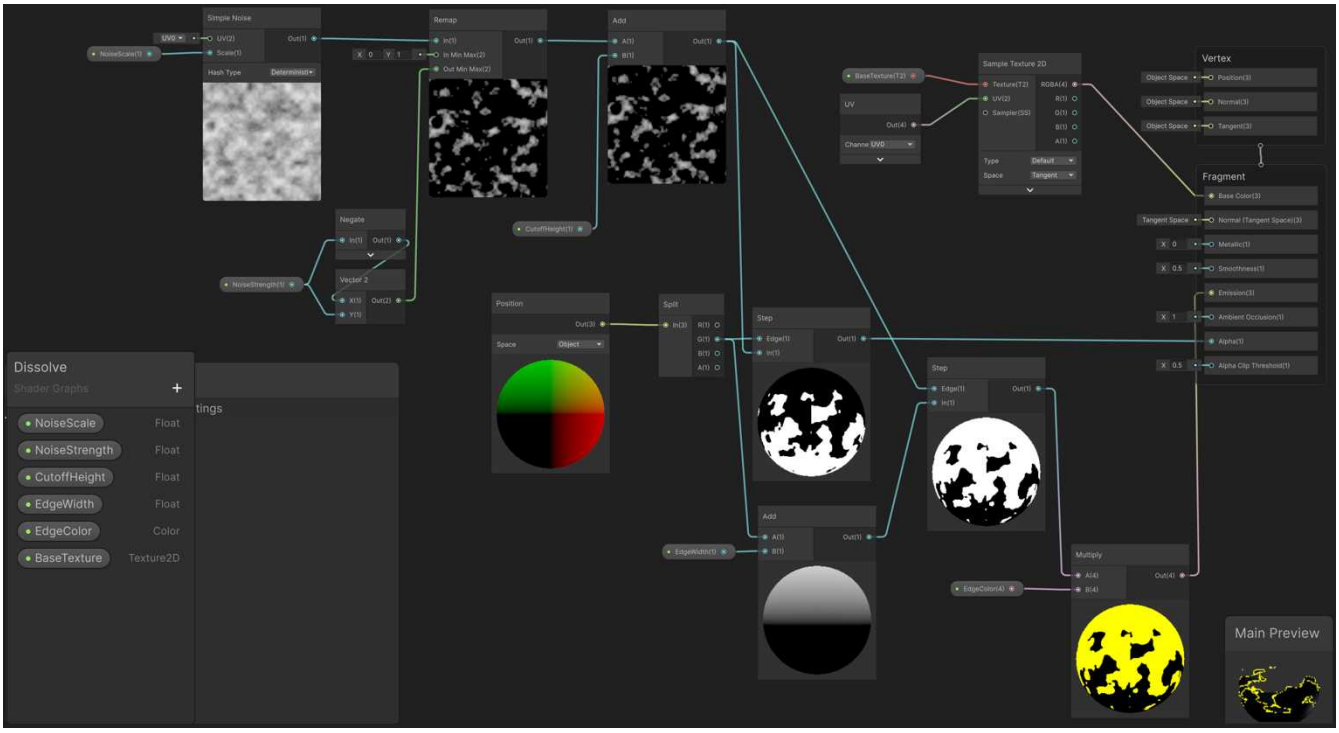
Dissolve Shader

A custom dissolve shader was created using Unity Shader Graph and applied to the spiders introduced into the environment. The shader is activated when a spider is burned, creating the effect of the creature gradually disappearing and the impression that it is burning instead of being instantly removed from the scene (Figure 12).

Dissolve Effect

The dissolve effect is driven by a procedural noise pattern generated with the Simple Noise node, see Figure 13. This noise is then remapped and combined with a value read from the height of the mesh, using the *object space* rather than its position in the world.

Figure 13
Dissolve shader graph.



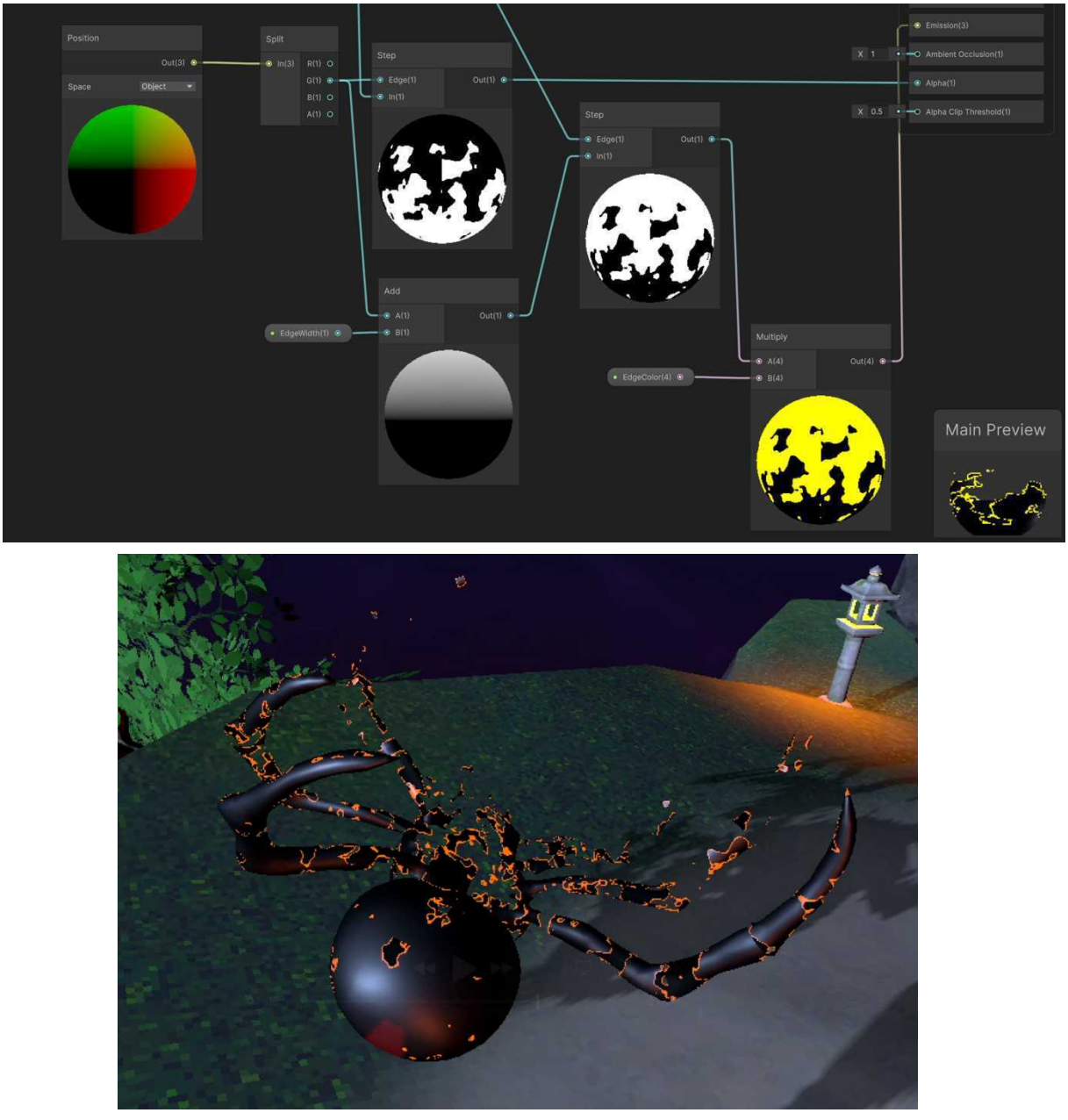
This is important because if the height were read from the world position instead, the effect would depend on where the object is placed in the scene, and the same shader would behave differently depending on the object's scale, rotation and location. Reading the height from the object's own space means the same material can be reused on every spider in the scene, regardless of where it stands or how large it is.

The result of this calculation is compared against a threshold value, called CutoffHeight. Wherever the noise value falls below this threshold, that part of the surface is discarded and becomes fully transparent using alpha clipping. As the CutoffHeight value increases, more of the spider disappears, creating the burning animation.

Edge Highlight

To make the effect look more like fire than a simple fade, a second calculation was added on top of the first. A slightly offset version of the same threshold is used to isolate a thin strip along the edge of the disappearing area, creating a margin between what is still visible and what has already dissolved. This strip is coloured using a bright HDR colour and connected to the material's emission, which makes it glow rather than just changing colour. This creates the bright glowing outline visible around the disappearing areas of the spider as it burns (Figure 14).

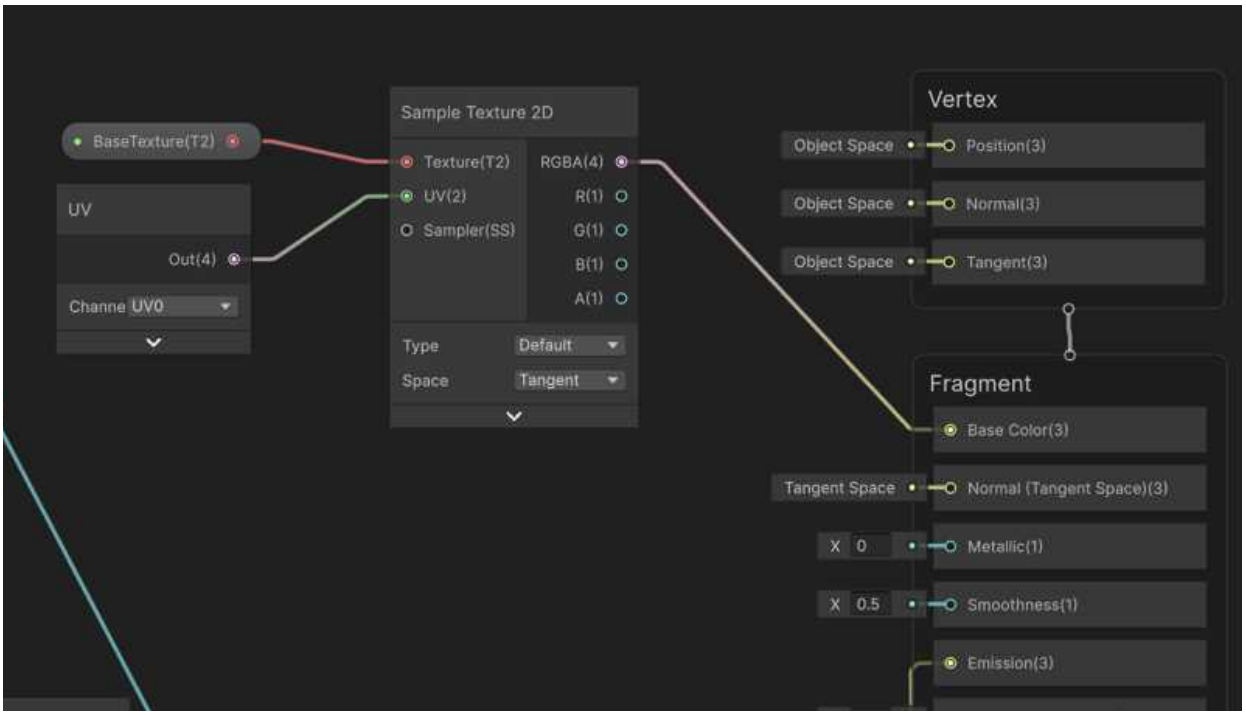
Figure 14
Emissive edge produced during the dissolve process – Graph and result.



Preserving the Spider Texture

As shown in figure 15, Since the spiders needed to keep their original black and red pattern instead of turning into a plain colour, a Texture2D parameter and a Sample Texture 2D node were connected to the Base Colour input, using the spider's original black widow texture. This allows the dissolve effect to appear on the spider's original appearance, instead of replacing it with a uniform colour.

Figure 15
Shader graph - Original black widow texture preserved during the dissolve effect.



Animating and Triggering the Burn

The shader itself does not animate automatically and only reacts to the current value of the CutoffHeight parameter. To create a gradual burning effect, CutoffHeight is animated through script using Mathf.Lerp(), see Figure 16. This gradually increases the dissolve threshold over time, creating a smooth transition from a fully visible spider to a completely dissolved one instead of an instant on/off effect.

Figure 16
Mathf.Lerp() – SpiderBurn script detail.

```
while (elapsed < m_burnDuration)
{
    elapsed += Time.deltaTime;
    float t = elapsed / m_burnDuration;
    float currentCutoff = Mathf.Lerp(m_cutoffStart, m_cutoffEnd, t);

    foreach (Material mat in m_materialInstances)
    {
        mat.SetFloat("_CutoffHeight", currentCutoff);
    }

    yield return null;
}
```

During testing, burning one spider caused every spider in the scene to dissolve at the same time. This happened because all spiders were sharing the same material instance. The issue was solved by creating a unique material instance for each spider using .material instead of .sharedMaterial. As a result, each spider can dissolve independently without affecting the others.

Figure 17
Unique instance material – SpiderBurn script detail.

```
0 references
private void Awake()
{
    // Grab a unique copy of the material for every renderer up front,
    // so I can safely animate CutoffHeight on this spider only.
    foreach (Renderer r in m_rendererstoBurn)
    {
        if (r != null)
            m_materialInstances.Add(r.material);
    }
}
```

Moreover, the spider model downloaded for this project is made of several separate parts: the main body and six individual eye pieces, each with its own renderer. For the dissolve effect to look consistent, the same material had to be applied to all these parts, not just the body, otherwise the eyes would have stayed visible after the rest of the spider had already disappeared.

The burning sequence is triggered by gameplay. Once the player has gained the burning power from the chest, getting close enough to a spider plays its death animation first, followed shortly after by the dissolve effect, so the spider is seen falling before it starts to burn away.

Figure 18
Burn power control – SpiderBurn script detail.

```
0 references
private void OnTriggerEnter(Collider other)
{
    // Ignore new triggers if this spider is already burning.
    if (m_isBurning) return;

    if (other.CompareTag("Player"))
    {
        PlayerPowers player = other.GetComponent<PlayerPowers>();

        if (player != null && player.CanBurnSpiders)
        {
            // Player has the power from the chest: start the burn sequence.
            StartCoroutine(BurnRoutine());
        }
        else
        {
            // Player got close without the power yet
            // so just play ascare reaction.
            if (m_animator != null)
            {
                m_animator.SetTrigger("Scare");
            }
        }
    }
}
```

Special Effects

The chest already present in the original scene opens automatically when the player approaches, but originally had no visual, audio, or gameplay consequence beyond the animation itself. To make this interaction more meaningful, a particle effect, a concentrated light, and a sound effect were added (Figure 19). The chest was also connected to the burning power described in the previous section.

Figure 19

Chest opening sequence.

Particle Effect and Light

When the chest opens, a particle effect and a concentrated point light are triggered to mark the moment. Both were initially attached directly to the trigger event, but this caused them to appear as soon as the player entered the trigger zone, before the chest was even open. This was corrected using Invoke(), see Figure 20, which delays the activation of both effects until the opening animation has finished based on its actual length, making the sequence feel more natural and visually coherent.

Figure 20

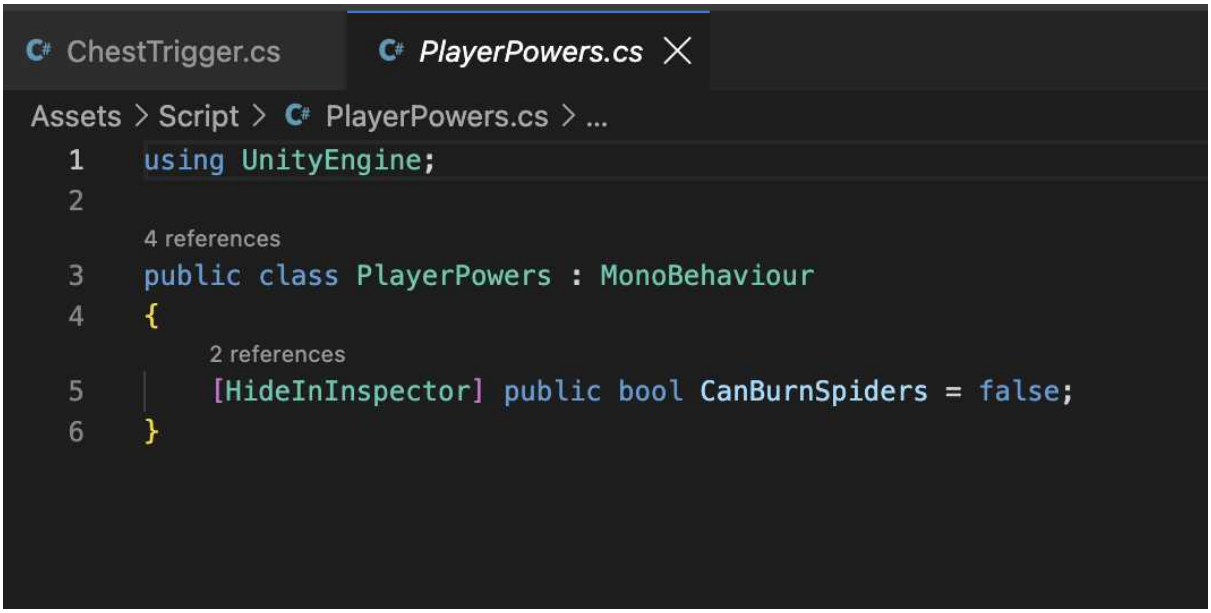
ChestTrigger script

```
            PlayerPowers player = other.GetComponent<PlayerPowers>();
            if (player != null)
            {
                player.CanBurnSpiders = true;
            }
            // Schedule the burn effects to play after the animation has finished
            Invoke(nameof(PlayBurnEffects), animLength);
        }
    }
    // Method to play burn effects after the chest opening animation
    1 reference
    private void PlayBurnEffects()
```

Granting the Burning Power

Opening the chest does not only trigger visual effects. It also grants the player the ability to burn spiders. This is handled through the PlayerPowers script (Figure 21) attached to the player, which stores whether the burning power has been acquired. When the chest opens, the script retrieves the player's PlayerPowers component and sets CanBurnSpiders to true. Every spider in the scene checks this value before reacting to the player's approach.

Figure 21
PlayerPowers Script.



Sound

An AudioSource component (Syntoca, 2023) was added to the chest and configured to play an opening sound effect. The sound starts at the same moment the chest animation begins, helping reinforce the interaction and making the opening sequence more immersive.

Conclusion

The result, see Figure 22, transforms the previous environment into a coherent, horror-themed experience through the combination of the elements explained above. Rather than treating these as three separate additions, each one was built to reinforce the others: the cold moonlight and unsettling red glow set the tone for the island, the chest ties the special effect directly to gameplay by granting the player a new ability, and the dissolve shader gives that ability a visual identity of its own, tying the whole experience together around a single central mechanic.

The dissolve effect itself is never triggered manually or shown in isolation. It only activates as a direct consequence of the player's actions, through the chest, the trigger, and the spider's animation sequence. This shows the shader working as part of a pipeline (chest → power → spider → animation → dissolve) driven by gameplay, rather than as an isolated visual effect.

Figure 22

Final scene overview.

Overall, the project demonstrates how environment design, lighting, shaders, particle effects, animation, and scripting can work together to transform a simple scene into a more immersive and visually cohesive experience.

References

CodeDog0. (2024). *Ocean & lake shaders | URP* [Shader asset]. Unity Asset Store. https://assetstore.unity.com/packages/vfx/shaders/ocean-lake-shaders-urp-303983

Dante's Anvil. (2022). *Fantasy spider* [3D model]. Unity Asset Store. https://assetstore.unity.com/packages/3d/characters/animals/insects/fantasy-spider-236418

Rpgwhitelock. (2024). *AllSky Free - 10 Sky / Skybox Set* [Skybox asset]. Unity Asset Store. https://assetstore.unity.com/packages/2d/textures-materials/sky/allsky-free-10-sky-skybox-set-146014

Syntoca. (2023). *Door, Cabinets & Lockers (Free)* [Audio file]. Unity Asset Store. https://assetstore.unity.com/packages/audio/sound-fx/foley/door-cabinets-lockers-free-257610

Tanuki Digital. (2019). *Stone Lantern Pack* [3D model]. Unity Asset Store. https://assetstore.unity.com/packages/3d/environments/historic/stone-lantern-pack-139776